
SOFTWARE DEVELOPMENT II PROJECT

OVERALL REPORT

Emmanouilidis Emmanouil 4669

Skretas Nikolaos 4493

TABLE OF CONTENTS

Introduction	3
First Contact	3
Legacy System - Source code Notes	3
Legacy System - Documentation Notes	4
Legacy System - Use Cases	6
Legacy System Installation Notes	8
Initial Understanding and Detailed Model Capture	13
Tests	16
Reengineered/Extended Design of the Legacy System	17

INTRODUCTION

The main objective of this project was to understand the given legacy code and then refactor it to solve various code related problems. This process helped us understand the value of clean code and how such code can be easily extended.

FIRST CONTACT

LEGACY SYSTEM - SOURCE CODE NOTES

- **Technology Stack:** Java 17, Spring Boot 3.4.1, Thymeleaf, MySQL (Connector), Spring Data JPA, Spring Security.
- **Follows the MVC architecture**
- **Field Injection:** extensive use of @Autowired on fields.
- **Service Layer:** Exists (UserService, UserServiceImpl), separating some business logic from controllers.
- **Domain Layer:** User, Student, Professor, Company hierarchy exists.
- **God Controller:** TraineeshipAppController contains logic for ALL user roles (Student, Professor, Company, Committee). This violates the Single Responsibility Principle. It should be split into role-specific controllers (e.g., StudentController, CompanyController).

Reengineering Feasibility Assessment

- **Very Doable because :**
 - The codebase is relatively small, making it manageable to refactor.
 - The current state (no tests, God Controller, confusing naming) will severely hamper future development and maintenance.

1. Introduction & Project Scope

- Clearly explains the purpose of the system: managing traineeship positions, assignments, supervision, and evaluations.
- Identifies all main stakeholders (students, companies, professors, committee), which is essential for understanding system roles.
- Useful as a high-level overview when onboarding new developers or maintainers.

2. Functional Requirements / User Stories

- Defines all system functionality through clear user stories (US1–US21).
- Helps developers:
 - Understand what features must exist
 - Trace code and database entities back to requirements
 - Validate whether the implemented system is complete
- Extremely useful for maintenance, debugging, and feature extension.

3. Non-Functional Requirements

- Defines architectural constraints and quality goals, especially:
 - Maintainability
 - Extensibility
 - Low coupling / high cohesion

- Explicitly recommends design patterns (Strategy, Template Method), which helps future developers understand *why* certain abstractions exist.
- Explains the reasoning behind configurable search and allocation strategies.

4. Development Process & Deliverables

- Explains how the system was originally developed (Scrum, sprints).
- Useful for understanding:
 - Why some parts are incremental or incomplete
 - Why certain design decisions prioritize extensibility over simplicity
- Sprint report expectations help interpret accompanying documentation.

5. Technology Stack & Constraints

- Specifies core technologies:
- Helps maintainers understand:
 - Why certain frameworks are used
 - Which testing tools are expected
- Useful for environment setup and consistency.

6. Missing or Weak Documentation Areas

- No detailed architecture diagrams

- No database schema / ER diagrams

Why this matters

- Makes readability harder.
- Indicates the system relies heavily on code-as-documentation.

Overall Impression

- The documentation is strong on requirements and design intent.
- It is less strong on operational and technical details.

LEGACY SYSTEM - USE CASES SPECIFY THE USE CASES OF THE APPLICATION.

Use case ID	UC1-Account Creation (Register)
Actors	User
Pre conditions	User is not registered.
Main flow of events	1. The use case starts when the user navigates to /register. 2. The system displays the registration form. 3. The user enters username, password, and selects a role. 4. The user submits the form (/save). 5. The system checks if the user exists. 6. If not, the system saves the new user and redirects to login.
Post conditions	New user account is created.

Use case ID	UC2-Account Login
--------------------	-------------------

Actors	Registered User
Pre conditions	User has an account
Main flow of events	1. The use case starts when the user navigates to /login. 2. The user enters credentials. 3. The system operates Spring Security authentication. 4. On success, the CustomSecuritySuccessHandler redirects the user to their role-specific dashboard.
Post conditions	User is authenticated and authorized.

Use case ID	UC3-Logout
Actors	Authenticated User
Pre conditions	User is logged in
Main flow of events	1. The use case starts when the user clicks "Logout". 2. The system executes Spring Security logout flow. 3. The session is invalidated. 4. The user is redirected to the home page.
Post conditions	User session is terminated.

Use case ID	UC4-Student Registration/Profile
Actors	Student
Pre conditions	User is logged in as a Student.
Main flow of events	1. The use case starts when the user navigates to /student/profile. 2. The system retrieves the logged-in user's username. 3. The system checks if a Student profile exists. 4. If not, a new Student object is created. 5. The system displays the profile form (student/profile). 6. The student fills in details (University ID, interests, skills, preferred location) and submits. 7. The system saves the profile and marks the student as lookingForTraineeship = true.
Alternative flow 1	The user has entered the wrong password and cannot login.
Alternative flow 2	The user has entered the wrong username and cannot login.
Post conditions	Student profile is updated and application for traineeship is active.

Use case ID	UC5-Find Positions for Student
Actors	Committee Member
Pre conditions	Committee member is logged in.
Main flow of events	1. The use case starts when the user selects a student from the traineeship applications list (/committee/list_traineeship_applications). 2. The user selects a search strategy (e.g., "Interests" or "Location") and clicks "Find Positions".

	3. The system executes the selected PositionsSearchStrategy. 4. The system calculates and returns a list of matching TraineeshipPosition objects. 5. The results are displayed to the user (committee/available_positions).
Post conditions	A list of matching positions is displayed.

Use case ID	UC6-Assign Position to Student
Actors	Committee Member
Pre conditions	Matching positions are displayed (Post-condition of UC2).
Main flow of events	1. The use case starts when the user selects a position from the search results (/committee/assign_position). 2. The system retrieves the Student and TraineeshipPosition entities. 3. The system links the student to the position and sets isAssigned = true. 4. The system updates the student status to lookingForTraineeship = false. 5. The system saves the changes.
Post conditions	Student is assigned to a position. Position is marked as assigned.

Use case ID	UC7-Company Registration/Profile
Actors	Company

Pre conditions	User is logged in as Company.
Main flow of events	1. The use case starts when the user navigates to /company/profile. 2. The system checks if a Company profile exists. 3. If not, a new Company object is initialized. 4. The user fills in details (Name, Location) and submits. 5. The system saves the company profile.
Post conditions	Company profile is updated.

Use case ID	UC8-Add Traineeship Position
Actors	Company
Pre conditions	Company profile exists.
Main flow of events	1.The use case starts when the user navigates to /company/show_position_form. 2. The system displays an empty position form. 3. The user enters position details (Title, Description, etc.) and submits. 4. The system links the position to the company. 5. The system saves the new position.
Post conditions	A new traineeship position is created and linked to the company.

Use case ID	UC9-List Available Positions
Actors	Company
Pre conditions	Company has created positions.
Main flow of events	1. The use case starts when the user clicks to list available positions (/company/list_available_positions). 2. The system retrieves the company's positions. 3. The system filters for positions where isAssigned == false. 4. The system displays the list.
Post conditions	Available positions are listed.

Use case ID	UC10-Professor Registration/Profile
Actors	Professor
Pre conditions	User is logged in as Professor.
Main flow of events	1. The use case starts when the user navigates to /professor/profile. 2. The system checks if a Professor profile exists. 3. If not, a new Professor object is initialized. 4. The user fills in details (Name, Interests) and submits. 5. The system saves the professor profile.

Post conditions	Professor profile is updated.
------------------------	-------------------------------

Use case ID	UC11-Assign Supervisor
Actors	Committee Member
Pre conditions	Position is assigned to a student (UC3).
Main flow of events	1. The use case starts when the user initiates supervisor assignment for a position (/committee/assign_supervisor). 2. The user selects an assignment strategy (e.g., "Interests" or "Load"). 3. The system executes the SupervisorAssignmentStrategy. 4. The system updates the position with the assigned professor.
Post conditions	Supervisor is assigned to the traineeship position.

Missing Functionality (Committee):

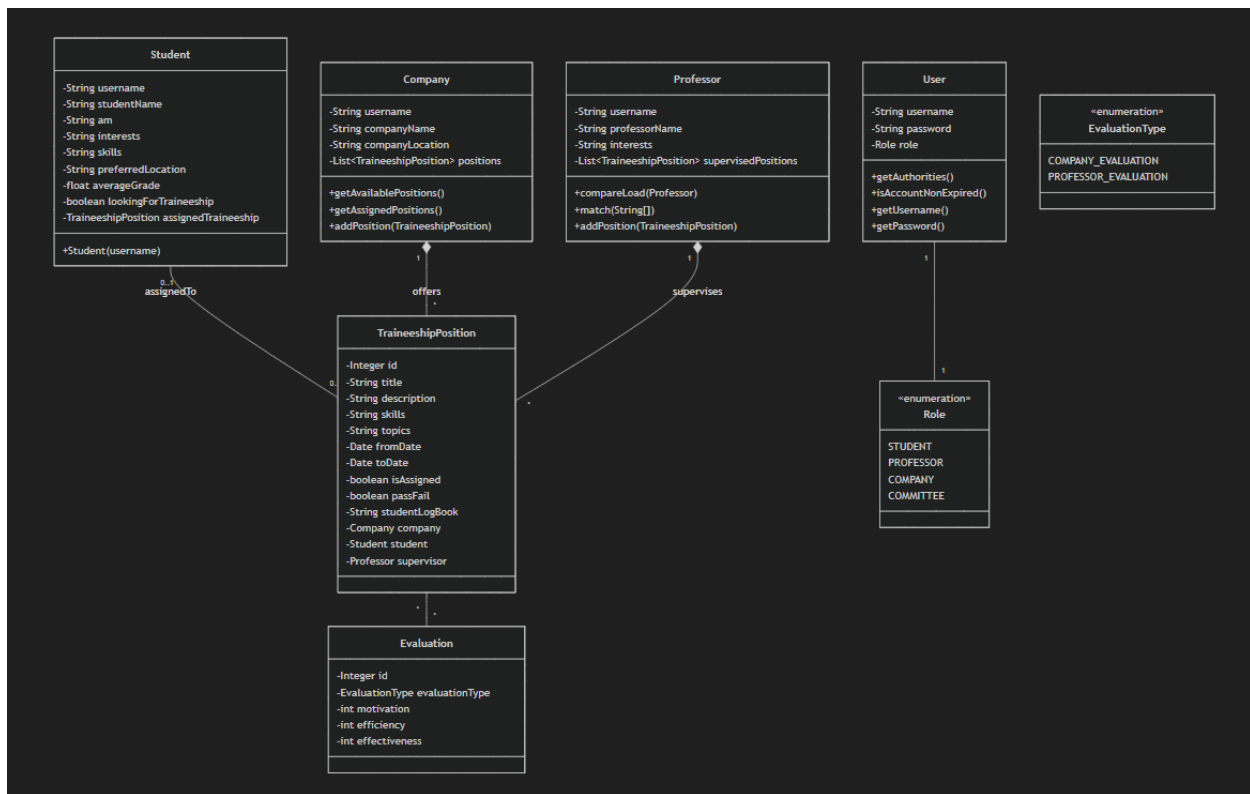
- **US20 (List In-Progress Traineeships):** Unimplemented.
- **US21 (Monitor/Complete Traineeship):** Unimplemented

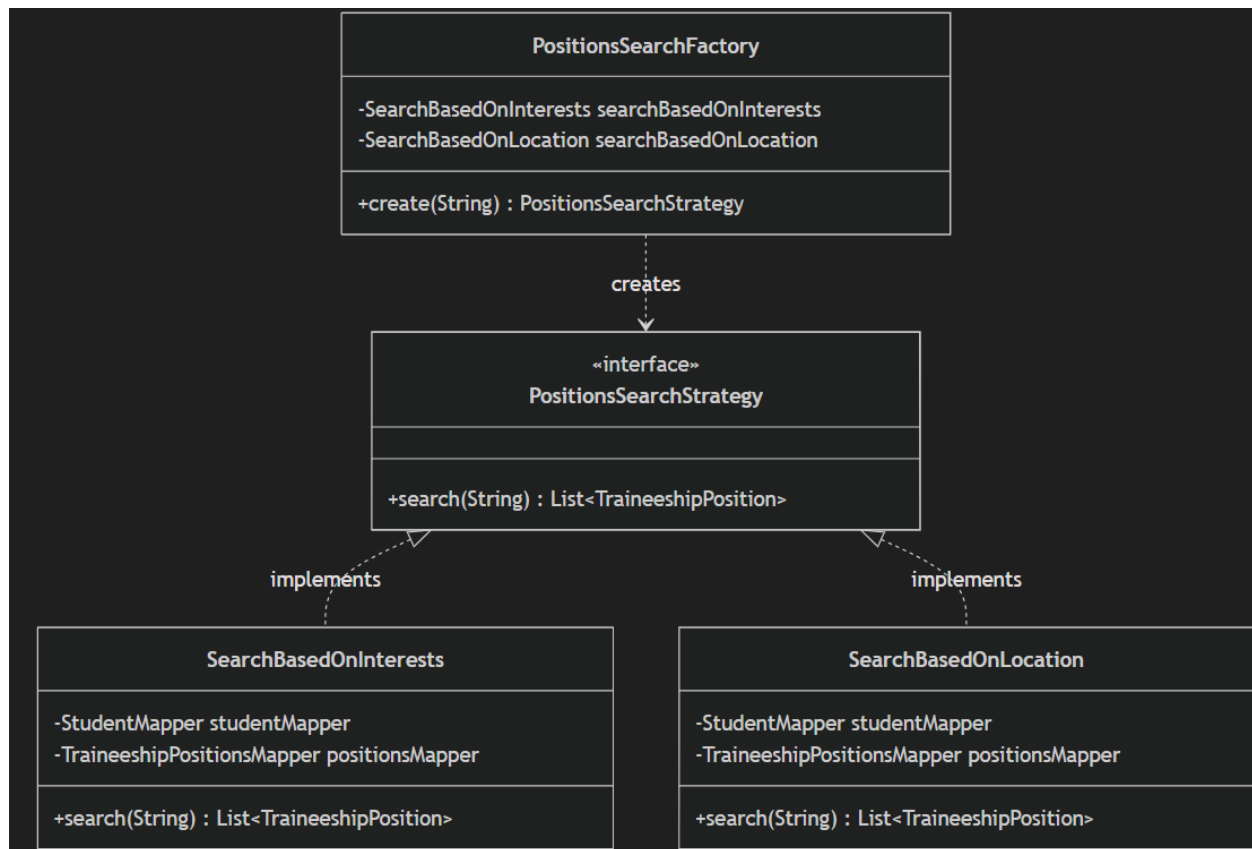
These were implemented to check if the program runs correctly.

The following specifications were identified after inspecting the configuration:

- **Libraries & Frameworks:**
 - Spring Boot: 3.4.1
 - Java Version: 17 (Project setting), Built with 21.0.2.
 - Template Engine: Thymeleaf spring-boot-starter-thymeleaf
 - Security: Spring Security 6 (spring-boot-starter-security)
 - Data Access: Spring Data JPA (spring-boot-starter-data-jpa)

INITIAL UNDERSTANDING AND DETAILED MODEL CAPTURE





Class Responsibilities

Domain Layer

- **User:** Represents authentication credentials and implements Spring Security's `UserDetails`. Stores username, password, and role for access control.
- **Student:** Extends user profile with academic details (AM number, interests, skills, location preference). Maintains a reference to assigned traineeship.
- **Professor:** Manages academic supervisor profile with interests. Tracks supervised positions and provides methods for load comparison and topic matching.
- **Company:** Represents organization offering traineeships. Manages list of positions and provides filtering methods for available vs. assigned positions.

- **TraineeshipPosition:** Core entity linking students, companies, and professors. Contains position details, dates, and assignment status.
- **Evaluation:** Stores performance metrics (motivation, efficiency, effectiveness) for either company or professor evaluations.

Controller Layer

- **TraineeshipAppController:** **God Class** handling ALL role-specific operations (Student, Professor, Company, Committee).
- **AuthController:** Manages registration and login flows. Delegates user creation to UserService.

Service Layer

- **UserService/UserServiceImpl:** Handles user registration, credential validation, and role-based entity creation. Implements UserDetailsService for Spring Security integration.

Repository Layer (Mappers)

- **StudentMapper, ProfessorMapper, CompanyMapper, TraineeshipPositionsMapper, UserMapper:** Spring Data JPA repositories providing CRUD operations and custom query methods.

Strategy Pattern

- **PositionsSearchStrategy:** Interface for position search algorithms. Implementations filter positions by student interests or preferred location.
- **SupervisorAssignmentStrategy:** Interface for supervisor assignment algorithms. Implementations assign based on topic matching or workload balancing.
- **Factories:** Create appropriate strategy instances based on runtime selection.

TESTS

Tests were implemented for the original and the refactored code.

1. Domain Object Tests (Unit Tests)

- CompanyTest, ProfessorTest, StudentTest, UserTest : Verify the creation and state management of human/organizational actors.
- TraineeshipPositionTest : Tests the behavior of the position entity, including status updates.
- EvaluationTest, EvaluationTypeTest : Correctness of evaluation logic and score calculations.
- RoleTest : Verifies user role management.

2. Service Layer Tests (Unit Tests with Mocking)

These tests focus on the business logic of the application. They use **Mockito** to mock dependencies (DAOs/Mappers), ensuring that the service methods behave correctly in isolation without accessing the actual database.

- StudentServiceTest: Verifies student-specific logic like retrieving profiles or saving logbooks.
- CompanyServiceTest : Tests company operations such as posting new positions.
- ProfessorServiceTest : Verifies professor actions, including supervising assignments.
- TraineeshipCommitteeServiceTest : Tests administrative functions for the committee.

3. Controller Layer Tests (Integration/Web Tests)

These tests verify the web layer using **Spring WebMvcTest**. They ensure that HTTP requests are handled correctly, the right views are returned, and the model attributes are properly populated.

- StudentControllerTest: Simulates HTTP requests (GET/POST) to student endpoints to verify the controller's interaction with the Service layer and the resulting UI response.

4. Data Access Layer Tests (Integration Tests)

These tests verify that the application interacts correctly with the database.

- `TraineeshipPositionsMapperTest`: Uses an in-memory database (H2) to test custom JPA queries, specifically ensuring that the new skill-based search filters retrieve the correct data from the database.

REENGINEERED/EXTENDED DESIGN OF THE LEGACY SYSTEM

Refactoring Task 1 For this task the monolithic controller was split into separate, focused controllers based on the actors they serve:

- `AuthController`: Handles authentication and login/logout logic.
- `StudentController`: Manages student-specific features (profile, logbook).
- `CompanyController` : Handles company actions and offerings.
- `ProfessorController` : Manages professor-specific tasks.
- `TraineeshipCommitteeController` : Handles administrative committee functions.

This Separation of Concerns makes the code more modular, maintainable, and easier to test, adhering to the Single Responsibility Principle.

Refactoring Task 2 (Duplicate Code) For this task, the **Template Method** design pattern was employed to resolve code duplication across the search and assignment strategies.

- Abstract class `AbstractPositionsSearchStrategy` was introduced.
- This class defines the skeleton of the algorithm in a final method (the template), managing the common workflow

- The specific variant logic (e.g., filtering by skills vs. interests) is deferred to abstract sub-methods (doSearch, findSupervisor) that concrete subclasses implement. This ensures the core process is defined in one place, reducing redundancy and minimizing the risk of inconsistent behavior.

Refactoring Task 3 For this task the SearchBasedOnSkills class was implemented to extend the search capabilities of the application. The doSearch method was developed within this class to parse the student's skills and filter TraineeshipPosition objects accordingly, strictly implementing the Search Strategy pattern to decouple the search logic from the controller.

Refactoring Task 4 (Extract Service Classes) - For this task, the **Extract Class** refactoring was applied to the service layer to decompose the monolithic UserService class. The original UserService had accumulated too many responsibilities, handling operations for all user types. It was refactored into separate, domain-specific service classes:

- **CompanyService/CompanyServiceImpl:** Handles company-specific business logic and operations.
- **ProfessorService/ProfessorServiceImpl:** Manages professor entities and supervisor assignment operations.
- **StudentService/StudentServiceImpl:** Manages student entities and application-related logic.
- **TraineeshipCommitteeService/TraineeshipCommitteeServiceImpl:** Handles administrative committee functions and decision-making processes.

This refactoring adheres to the **Single Responsibility Principle** and improves code maintainability, testability, and modularity by ensuring each service has a clear, focused responsibility aligned with its corresponding domain entity.

Implemented User Stories

The following user stories were successfully implemented as part of the traineeship management application:

Student Features

- **US6:** As a student I want to be able to fill in my traineeship logbook to report what I do in the traineeship that has been assigned to me.

Company Features

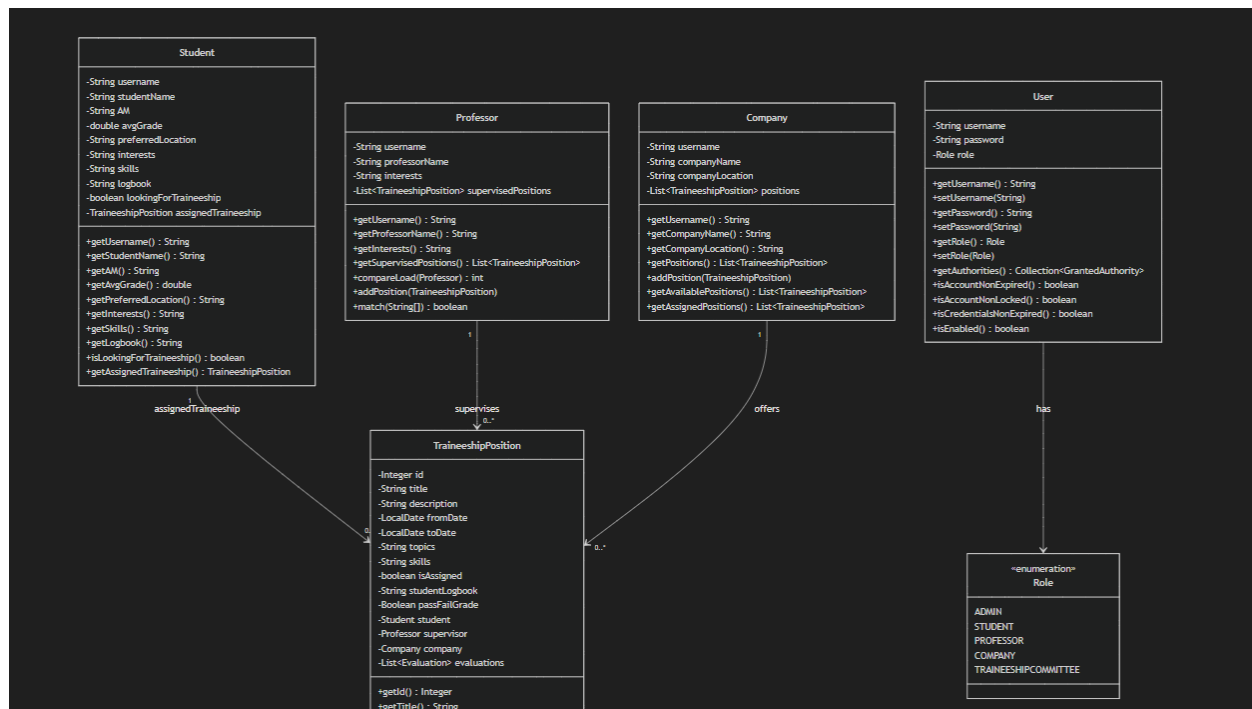
- **US9:** As a company we want to have access to the list of traineeships positions that have been assigned to students.
- **US11:** As a company we want to be able to delete a traineeship position that is no longer available.
- **US12:** As a company we want to be able to fill in an evaluation for a traineeship that is in progress. The evaluation rates the student motivation, effectiveness and efficiency on a scale from 1 (bad) to 5 (perfect).

Professor Features

- **US14:** As a professor I want to have access to the list of traineeship positions that I supervise.
- **US15:** As a professor I want to be able to fill in an evaluation for a traineeship that I supervise. The evaluation rates the student motivation, effectiveness and efficiency on a scale from 1 (bad) to 5 (perfect), and evaluates the hosting company in terms of the facilities provided and the guidance of the trainee student on a scale from 1 (bad) to 5 (perfect).

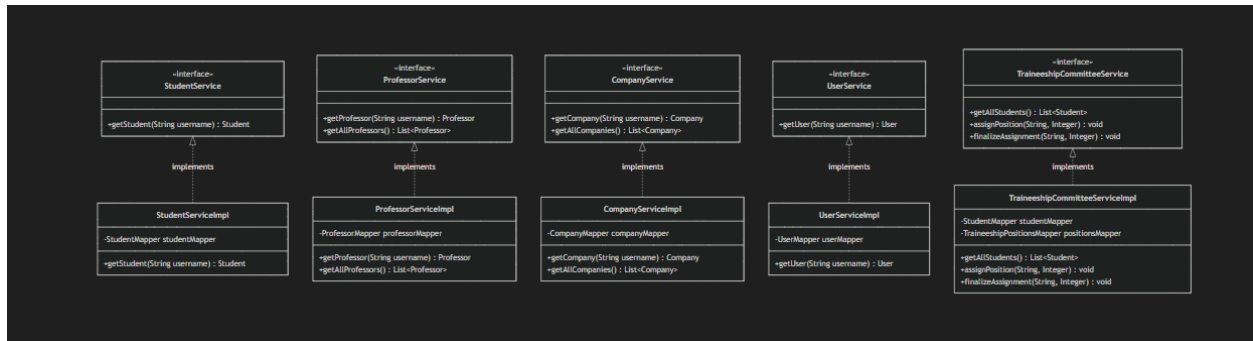
1. Domain Layer Class Diagram

The domain layer contains the core entities representing the business model.



2. Service Layer Class Diagram

The service layer implements business logic and orchestrates domain operations.



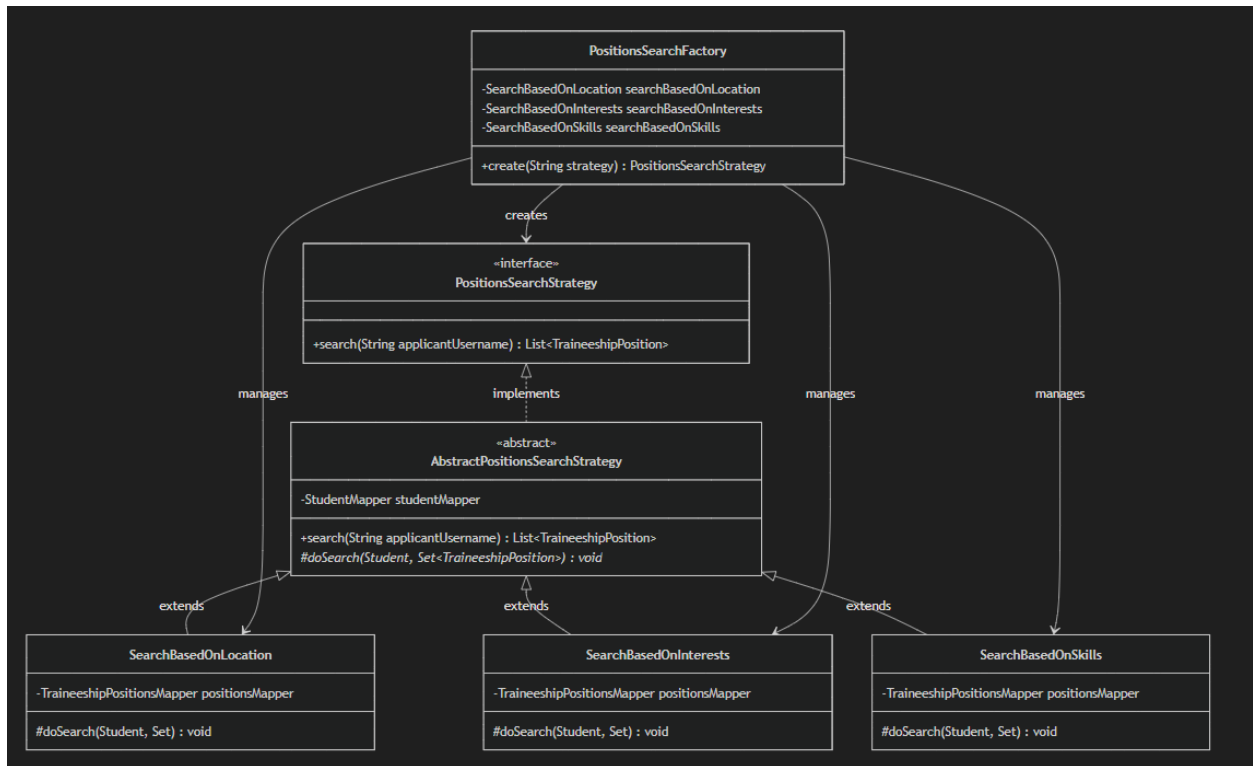
3. Controller Layer Class Diagram

The controller layer handles HTTP requests and implements REST API endpoints.



4. Strategy Pattern - Position Search

The system implements the Strategy pattern for flexible position searching algorithms.



Key Design Patterns and Principles

1. Layered Architecture

The system follows a clear layered architecture:

- Controllers handle HTTP requests and responses
- Services contain business logic
- Mappers handle data persistence
- Domain entities represent the business model

- Clear separation of concerns across layers

2. Strategy Pattern

- **Position Search Strategies:** Allow flexible algorithms for matching students with positions

The Factory pattern is used to create appropriate strategy instances.

3. Template Method Pattern

Strategy hierarchy uses Template Method:

- Abstract base classes define the algorithm structure
- Concrete classes implement specific steps
- Common logic is shared in the base class

4. Single Responsibility Principle

- Each class has a focused responsibility
- Controllers handle HTTP, Services handle business logic
- Separation enables independent evolution of components

!!To add data to the tables and check the correctness of the implemented User Stories, you can run the myy803_traineeship_app-static.sql script. All passwords are "committee"!!